Тема 8. Управление пакетами - NPM.

хекслет колледж

Цель занятия:

Научиться структурировать проект с помощью файла package.json, управлять внешними библиотеками через NPM и эффективно организовывать рабочее окружение для создания переносимых и масштабируемых приложений.

хекслет колледж

Учебные вопросы:

1. Что такое NPM (Node Package Manager)?
2. Инициализация проекта: npm init
3. Файл package.json - Паспорт проекта
4. Директория node_modules и package-lock.json
5. Команды управления пакетами

хекслет колледж

1. Что такое NPM?

NPM **(Node Package Manager)**- это менеджер пакетов для JavaScript, который устанавливается автоматически вместе с Node.js.

На сегодня это самый большой реестр программного обеспечения в мире (более 2 миллионов пакетов).

NPM состоит из трех независимых частей:

1. Веб-сайт (npmjs.com): Площадка для поиска библиотек, чтения документации и проверки популярности пакетов.

2. Реестр (Registry): Огромная база данных кода, откуда скачиваются пакеты.

3. Интерфейс командной строки (CLI): Та самая команда npm в терминале, с помощью которой мы управляем проектом.

кекслет колледж

Зачем он нужен программисту?

Главная концепция NPM - модульность.

Вместо того чтобы писать всё с нуля, вы скачиваете «пакеты» (готовые модули), решающие конкретные задачи.

Примеры из практики:

- Нужно работать с датами? Скачиваем date-fns.
- Нужно раскрасить вывод в консоли? Ставим chalk.
- Нужно парсить музыку? Берем готовый инструмент для работы с аудио-тегами.

кекслет колледж

Как это работает: Роли NPM

Роль	Описание
Установщик	NPM скачивает нужный код и кладет его в ваш проект.
Менеджер версий	Он следит, чтобы вы использовали стабильную версию библиотеки, и помогает обновляться.
Диспетчер зависимостей	Если библиотека А зависит от библиотеки Б, NPM сам скачает обе.

кекслет колледж

Глобальные и локальные пакеты

NPM позволяет устанавливать инструменты двумя способами:

1. Локально (по умолчанию): Пакет устанавливается только в папку вашего текущего проекта. Это стандарт для 99% случаев.
   - Пример: Библиотека для обработки запросов.

2. Глобально (флаг -g): Пакет устанавливается в систему как полноценная программа.
   - Пример: npm install -g nodemon (чтобы запускать его из любой папки).

хекслет колледж

Итог:

- NPM - это инструмент, который превращает ваш проект из набора файлов в структурированное приложение.
- Пакет (package) - это папка с кодом и файлом package.json, который описывает этот код.
- Основная ценность NPM - сообщество. Почти любую рутинную задачу в Node.js уже кто-то решил и выложил в виде пакета.

хекслет колледж

2. Инициализация проекта.

Этот этап превращает обычную папку с файлами в официальный Node.js-проект.

Без инициализации вы не сможете управлять зависимостями или автоматизировать запуск кода.

хекслет колледж

Инициализация проекта: npm init

Чтобы начать работу с NPM, нужно создать файл-манифест package.json. Это делается через команду инициализации в терминале.

Откройте терминал в папке вашего будущего проекта и введите:

```
npm init
```

хекслет колледж

После этого NPM начнет «интервью». Он задаст серию вопросов, чтобы заполнить данные о проекте:

- package name: Имя проекта (по умолчанию - название папки).
- version: Версия (начинается с 1.0.0).
- description: Краткое описание (например, «Интернет-магазин»).
- entry point: Главный файл приложения (обычно index.js или app.js).
- test command: Команда для запуска тестов.
- git repository: Ссылка на ваш репозиторий.
- author: Ваше имя.
- license: Тип лицензии (обычно ISC или MIT).

По завершении в папке появится файл package.json.

кекслет колледж

Лайфхак: Быстрая инициализация

Если вы не хотите отвечать на все вопросы и согласны с настройками по умолчанию, используйте флаг -y (от англ. yes):

npm init –y

Это мгновенно создаст стандартный package.json, который потом можно отредактировать вручную.

хекслет колледж

Почему это важно?

- Без файла package.json ваш проект - это просто набор разрозненных файлов.
- Для системы: NPM не поймет, куда записывать информацию об установленных библиотеках.
- Для других разработчиков: Если вы передадите проект другу без этого файла, он не узнает, какие библиотеки нужно докачать, чтобы ваш код заработал.

хекслет колледж

Что делать, если ошиблись?

Файл package.json - это обычный текстовый файл в формате JSON.

Вы в любой момент можете открыть его в VS Code и изменить любое поле (например, поменять версию или имя автора).

хекслет колледж

File Edit Selection View Go ···

myproject [Administrator]

EXPLORER ···

MYPROJECT

{} package.json

{} package.json ×

{} package.json > ...

```
1  {
2    "name": "myproject",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
     ▷Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC"
12 }
13
```

ЖЕКСЕЛЕ КОЛЛЕКТИВ

Итог:

- npm init - команда, создающая «паспорт» проекта.
- Флаг -y экономит время, пропуская опросник.
- Главный результат: появление файла package.json, который станет фундаментом для всей дальнейшей работы с библиотеками.

хекслет колледж

3. Файл package.json - Паспорт проекта

Этот раздел объясняет, как управлять проектом и автоматизировать действия.

Если npm init - это процесс получения паспорта, то сам package.json - это документ, в котором записано всё: от имени «гражданина» до списка его друзей (зависимостей) и умений (скриптов).

хекслет колледж

package.json - это главный конфигурационный файл Node.js-проекта.

Он написан в формате JSON (JavaScript Object Notation), что делает его легкочитаемым и для человека, и для компьютера.

хекслет колледж

Основные блоки файла

1. Метаданные (Общая информация)

Эти поля описывают сам проект:

- "name": Название проекта (маленькими буквами, без пробелов).
- "version": Текущая версия. По стандарту SemVer (например, 1.0.0).
- "main": «Точка входа». Файл, который запустится первым, если кто-то подключит ваш проект как библиотеку.

кекслет колледж

2. Блок "scripts" (Автоматизация)
Это один из самых полезных разделов. Здесь можно задать алиасы (псевдонимы) для длинных консольных команд.
Пример:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

Как запустить? В терминале пишем npm run <название_скрипта>.
Исключение: Для команд start и test слово run можно опускать (npm start).

хекслет колледж

3. Блоки зависимостей (Список «друзей»)

Здесь NPM автоматически записывает библиотеки, которые вы устанавливаете:

- "dependencies": Пакеты, без которых программа не будет работать у пользователя (например, сервер express).
- "devDependencies": Пакеты, нужные только разработчику (например, nodemon для автоперезагрузки или тесты).

хекслет колледж

Правила формата JSON

Важно помнить:

1. **Двойные кавычки**: В JSON используются только ", одинарные ' приведут к ошибке.
2. **Запятые**: После последнего элемента в объекте запятая не ставится.
3. **Структура**: Это всегда один большой объект, заключенный в фигурные скобки {}.

кекслет колледж

Почему это «Паспорт»?

Если вы решите перенести свой проект на другой компьютер или отправить его через GitHub, вам не нужно пересылать тяжелую папку с библиотеками.

Достаточно передать один файл package.json. Новый разработчик введет команду:

```
npm install
```

И NPM сам прочитает «паспорт», увидит список нужных библиотек в разделах dependencies и скачает их актуальные версии.

хекслет колледж

Итог:

- package.json - это мозг и сердце проекта.
- Блок scripts позволяет не запоминать сложные команды запуска.
- Разделение на dependencies и devDependencies экономит ресурсы сервера (на сервер не устанавливается лишний «мусор» для разработки).

хекслет колледж

4. Директория node_modules и package-lock.json

Что такое node_modules?

Это «склад» вашего проекта. Когда вы вводите npm install, NPM скачивает код библиотек и физически кладет их в эту папку.

хекслет колледж

Особенности node_modules:

- Дерево зависимостей: Если вы установили библиотеку A, а ей для работы нужна библиотека B, то в node_modules появятся обе. Именно поэтому папка так быстро растет.
- Игнорирование (Git): Эту папку никогда не отправляют в репозиторий (GitHub). Она слишком тяжелая и легко восстанавливается.

Для этого создается файл .gitignore, куда просто записывается строчка node_modules/.

кекслет колледж

Зачем нужен package-lock.json?

Многие думают, что это копия package.json, но это не так. Если package.json - это список покупок (мне нужны «яблоки» версии 1.x), то package-lock.json - это чек из магазина, в котором записано всё до мельчайших деталей:

1. **Точная версия**: Не просто «версия 1.x», а конкретно 1.2.4.
2. **Хэш-сумма** (Integrity): Гарантия того, что скачанный код не был подменен или поврежден.
3. **Полная структура**: Список абсолютно всех вложенных зависимостей.

хекслет колледж

Почему это критически важно?

Представьте, что вы написали проект сегодня, а ваш коллега скачал его через месяц. За это время одна из библиотек обновилась и стала работать по-другому.

- Без package-lock.json у коллеги может всё сломаться.
- С package-lock.json NPM установит ему ровно те же версии байт в байт, которые были у вас.

хекслет колледж

Сравнение двух файлов

Характеристика	package.json	package-lock.json
Кто редактирует?	Программист вручную.	NPM автоматически.
Что содержит?	Основные зависимости и скрипты.	Полный «снимок» всех зависимостей.
Цель	Описать намерения и метаданные.	Гарантировать идентичность установки.

кекслет колледж

Итог:

- node_modules - это папка с физическим кодом. Мы её не трогаем руками и не пушим в Git.
- package-lock.json - это гарант того, что проект запустится одинаково на любом компьютере («State of the project»).
- Золотое правило: Если вы скачали проект из интернета и в нем нет папки node_modules, просто введите npm install.

кекслет колледж

5. Команды управления пакетами

Команды NPM позволяют не только скачивать код, но и гибко управлять его жизненным циклом.

Эти команды станут вашими основными инструментами на каждый день.

хекслет колледж

1. Установка пакетов (Install)

• Локальная установка:

npm install <name> (или кратко npm i <name>)

Скачивает пакет в node_modules и добавляет его в секцию dependencies вашего package.json.

• Для разработки (DevDependencies):

npm i <name> --save-dev (или -D)

Используется для инструментов, которые не нужны конечному пользователю (тесты, линтеры, nodemon).

• Установка всех зависимостей проекта:

npm install

Если вы скачали чужой проект, эта команда прочитает package.json и восстановит всё дерево библиотек.

кекслет колледж

2. Удаление пакетов (Uninstall)

Если библиотека больше не нужна, её нужно правильно «выписать» из проекта:

npm uninstall <name> (или npm un <name>)

Команда удалит папку из node_modules и сотрет упоминание о ней в package.json.

хекслет колледж

3. Глобальная установка (-g)

Некоторые пакеты - это не библиотеки для кода, а самостоятельные программы-утилиты.

npm i -g <name>

Такие пакеты устанавливаются в системную папку и доступны из любого места в терминале.

Пример: npm i -g npm (обновление самого менеджера пакетов).

хекслет колледж

4. Поиск и обновление

- npm outdated: Проверяет, не вышли ли новые версии для ваших библиотек.

- npm update: Обновляет пакеты до максимально допустимой версии (согласно правилам в package.json).

хекслет колледж

5. Специфические команды

- npm root -g: Показывает, где физически на вашем диске лежат глобальные пакеты.

- npm list: Выводит дерево всех установленных зависимостей в текущем проекте.

хекслет колледж

Шпаргалка по командам:

Задача	Команда
Создать проект	npm init -y
Установить библиотеку	npm i <название>
Установить инструмент разработки	npm i -D <название>
Восстановить проект из GitHub	npm i
Запустить свой скрипт	npm run <имя_скрипта>

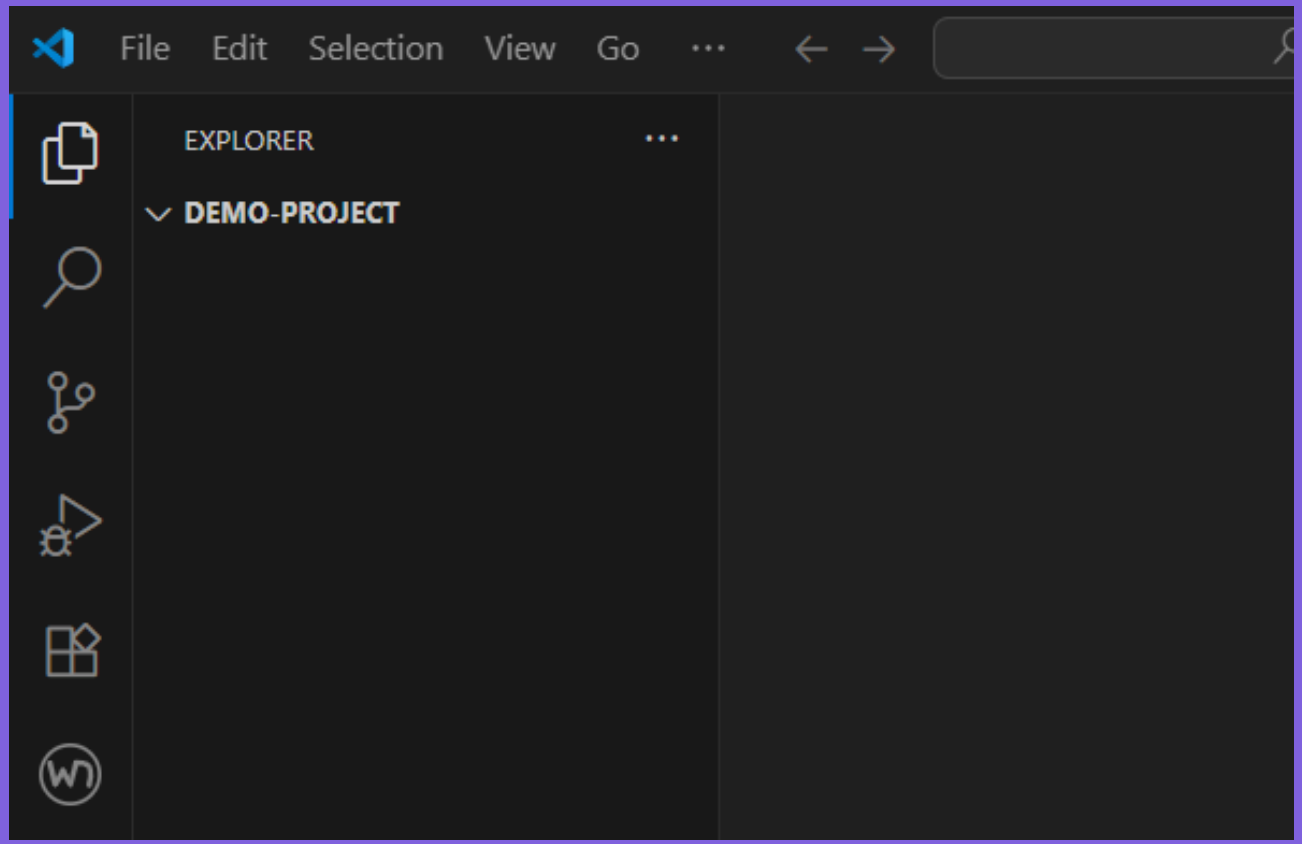
кекслет колледж

Практическая часть (Демонстрация)

1. Рождение проекта (Инициализация)

хекслет колледж

Создадим папку demo-project и зайдем в неё.



кекслет колледж

Выполним: npm init -y.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

  "license": "ISC"
}


PS C:\Users\user\Documents\nodeproject\demo-project> npm init -y
```

хекслет колледж

Появится package.json:

EXPLORER

DEMO-PROJECT

{} package.json

{} package.json

```
{
  "name": "demo-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

кекслет колледж

2. Установка зависимостей (dependencies)
Установим библиотеку chalk v4 (для раскраски консоли)

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

2 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Documents\nodeproject\demo-project> npm i chalk@4
```

кекслет колледж

В package.json появилась секция dependencies.

EXPLORER

DEMO-PROJECT
- node_modules
- index.js
- package-lock.json
- package.json

package.json × | index.js

package.json > ...

```
1  {
2    "name": "demo-project",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
     Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "dependencies": {
13     "chalk": "^4.1.2"
14   }
15 }
16
```

екслет колледж

Появилась папка node_modules (там не только chalk, но и его зависимости)

EXPLORER

- DEMO-PROJECT
  - node_modules
    - ansi-styles
    - chalk
      - source
      - index.d.ts
      - license
      - package.json
      - readme.md
    - color-convert
    - color-name
    - has-flag
    - supports-color
    - .package-lock.json
  - index.js
  - package-lock.json
  - package.json

package.json

node_modules > chalk > package.json > [ ] files

```
20      "keywords": [
39          "logging",
40          "command-line",
41          "text"
42      ],
43      "dependencies": {
44          "ansi-styles": "^4.1.0",
45          "supports-color": "^7.1.0"
46      },
47      "devDependencies": {
48          "ava": "^2.4.0",
49          "coveralls": "^3.0.7",
50          "execa": "^4.0.0",
51          "import-fresh": "^3.1.0",
52          "matcha": "^0.7.0",
53          "nyc": "^15.0.0",
54          "resolve-from": "^5.0.0",
55          "tsd": "^0.7.4",
```

кекслет колледж

Появился package-lock.json

EXPLORER

- DEMO-PROJECT
  - node_modules
    - ansi-styles
    - chalk
      - source
      - index.d.ts
      - license
      - package.json
      - readme.md
    - color-convert
    - color-name
    - has-flag
    - supports-color
    - .package-lock.json
  - index.js
  - package-lock.json
  - package.json

package-lock.json > ...

```
6     "packages": {
15      "node_modules/ansi-styles": {
26        "funding": {
28          }
29        },
30        "node_modules/chalk": {
31          "version": "4.1.2",
32          "resolved": "https://registry.npmjs.
33          "integrity": "sha512-oKnbhFyRIXpUuez
34          "license": "MIT",
35          "dependencies": {
36            "ansi-styles": "^4.1.0",
37            "supports-color": "^7.1.0"
38          },
39          "engines": {
40            "node": ">=10"
41          },
42          "funding": {
43            "url": "https://github.com/chalk/c
```

екслет колледж

3. Использование пакета. Создадим index.js и напишем пару строк:

```
const chalk = require('chalk');

console.log(chalk.blue('Привет, студенты!'));
console.log(chalk.green.bold('NPM работает успешно.'));
```

хекслет колледж

Запускаем через node index.js

```
PS C:\Users\user\Documents\nodeproject\demo-project> node index.js
Привет, студенты!
NPM работает успешно.
PS C:\Users\user\Documents\nodeproject\demo-project>
```

Мы не писали код для раскраски текста, мы просто взяли его из реестра NPM

кекслет колледж

4. Автоматизация (Scripts)

Добавим в package.json в раздел scripts:

"start": "node index.js"

package.json > ...

```
1  {
2    "name": "demo-project",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
     ▷ Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node index.js"
9    },
0    "keywords": [],
1    "author": "",
2    "license": "ISC",
3    "dependencies": {
4      "chalk": "^4.1.2"
5    }
```

кекслет колледж

Запустите через npm start.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

> demo-project@1.0.0 start
> node index.js

Привет, студенты!
NPM работает успешно.
PS C:\Users\user\Documents\nodeproject\demo-project> npm start
```

Теперь любой разработчик, скачав ваш проект, будет знать, как его запустить, не заглядывая внутрь кода.

хекслет колледж

5. Фокус с исчезновением (Восстановление)

Удалите папку node_modules физически!

Попробуйте запустить node index.js.

Программа «упадет» с ошибкой Error:

Cannot find module 'chalk'.

хекслет колледж

Теперь введите магическую команду:
npm install

Результат: Папка node_modules вернулась, всё снова работает.

Вот почему мы не передаем node_modules коллегам.
Список в package.json - это всё, что нам нужно».

хекслет колледж

6. Dev-зависимости

Установите nodemon как dev-зависимость: npm i -D nodemon

Убедитесь, что он попал в отдельный блок devDependencies

Добавьте скрипт
"dev": "nodemon index.js"

Выполните:
npm run dev

и посмотрите, как сервер сам перезагружается при правке текста.

хекслет колледж

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Documents\nodeproject\demo-project> npm i -D nodemon
```

```
{} package.json > {} scripts > dev
  "main": "index.js",
  ▷ Debug
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
```

екслет колледж

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\user\Documents\nodeproject\demo-project> npm run dev
```

```
const chalk = require('chalk');

console.log(chalk.blue('Привет, студенты!'));
console.log(chalk.green.bold('NPM работает успешно.'));
console.log(chalk.yellow.bold('сервер сам перезагружается при правке текста'));
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

[nodemon] starting `node index.js`
Привет, студенты!
NPM работает успешно.
сервер сам перезагружается при правке текста
[nodemon] clean exit - waiting for changes before restart
```

екслет колледж

Выводы:

NPM - Это не только инструмент для скачивания кода, но и инфраструктура, позволяющая управлять версиями и зависимостями проекта.

package.json - сердце проекта: В нем хранится вся информация: как проект называется, кто его автор, какие библиотеки ему нужны и какими командами его запускать.

Разделяй зависимости: Мы делим пакеты на те, что нужны для работы приложения (dependencies), и те, что нужны только программисту (devDependencies).

node_modules не для Git: Эта папка - физическое воплощение зависимостей. Её размер может быть огромным, поэтому мы передаем только «список покупок» (package.json), а папку игнорируем.

Воспроизводимость: Благодаря package-lock.json мы уверены, что через год или на другом компьютере проект соберется точно с теми же версиями библиотек, что и сегодня.

кекслет колледж

Контрольные вопросы

- Представьте, что вы случайно удалили файл package.json, но папка node_modules осталась. Сможете ли вы продолжать разработку и передать проект коллеге? Почему?
- В чем разница между запуском команд npm start и npm run dev? Почему в одном случае мы используем слово run, а в другом нет?
- Вы устанавливаете библиотеку для тестирования кода. С каким флагом её нужно установить и в какой секции package.json она должна появиться?
- Зачем нужен файл package-lock.json, если у нас уже есть список всех библиотек в package.json?
- Вы скачали проект с GitHub, в нем есть файлы index.js, package.json и .gitignore. Какую первую команду вы должны выполнить в терминале, чтобы проект заработал?

хекслет колледж